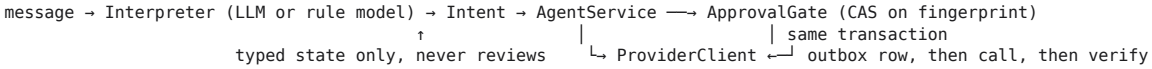
Errand agent: design note

Thesis: the model parses, code decides. The LLM turns one user message into a typed `Intent` and nothing else. A deterministic workflow owns state, every provider call goes through one client, and every write passes an approval gate, an outbox and a status check. The model never sees a write tool, so it cannot be talked into using one. This is the plan-then-execute pattern from *Design Patterns for Securing LLM Agents against Prompt Injections* (Beurer-Kellner et al., 2025): untrusted text can never choose an action. Reviews never reach the model, and provider names reach it only as neutral labels when they look like instructions (S42).



State model

`ErrandState` is one versioned JSON row per session (`sessions`, optimistic version check on save). It holds the task (`book | reschedule | cancel`), a phase (`idle → collecting → searching → choosing → awaiting_approval → executing → verifying → done`), and *slots*: each slot records its value, its source (`user`, `memory`, `history`) and the turn that set it. "What is missing" is a pure function of the slots (`policy.missing_fields`): service, area and date are required; time and budget are asked once and "no preference" is stored as a value, so a known fact is never asked again. A correction is a patch: `policy.apply_details` changes only the named slots and `policy.invalidations` decides what that voids downstream (a new date voids offers, hold and approval; a new budget only re-filters offers). A second request in the same message is queued and offered next, never dropped; ruled-out providers are a slot that filters offers. Once a booking is done, a detail change ("make it Wednesday") never silently starts a second booking: the agent asks once, move it or book another (S41).

Approval enforcement

An approval is a row: action, provider, slot, price, booking id, and `fingerprint = sha256(all of those)`. The hold id is deliberately not in the fingerprint, so re-holding the same slot at the same price is the same agreement. The prompt the user sees is rendered from the row. Granting needs a `UserTurnRef`, a token only the agent service can mint, plus an explicit yes in the user's own words (or a button press): if the model claims "confirm" for "what's your cancellation policy?", the grant is refused (scenario S31). Before writing, the executor recomputes the fingerprint from the *current* selection and hold quote, then runs, in one SQLite transaction, `UPDATE approvals SET status='consumed' WHERE id=? AND status='granted' AND fingerprint=?` and the outbox insert. Zero rows updated means no call. A changed provider, slot or price produces a different fingerprint, so the old approval is voided and a new one proposed (S03, S08, S09).

Idempotency and recovery

The key is derived, never random: `bk_` + sha256(approval id, hold id); `cx_`/`rs_` + approval id. Retries and restarts reuse it, and the mock honours keys server-side (a repeat returns the stored result; the same key with a different request is rejected, as Stripe does). Every provider outcome maps to one class: `UNKNOWN_OUTCOME` (timeout), `RETRYABLE` (503), `OPTION_GONE` (slot taken or conflict), `HOLD_GONE`, `REQUEST_INVALID`, `UNEXPECTED`. A timeout is never a failure: the executor asks `get_booking_status`, and retries with the same key only on `not_found`, with capped full-jitter backoff. Success is reported only after status says `booked`, even on a plain OK. Retries happen in one layer only. At the start of every turn, and when the console reconnects, any outbox row in `SENT` or `UNRESOLVED` is reconciled before the user's message is handled (S22, S23).

Memory

Facts live in `memory_facts` with `source` (stated or inferred), `status`, `confirmed_at` and `evidence`. Writes need a `UserTurnRef`, so a review cannot write memory (S05). Inference (the same technician booked twice) creates an unusable `pending_confirmation` fact; the agent asks once and uses it only after a yes (S20). Forgetting marks the fact forgotten and writes a tombstone, which blocks re-inference from booking history; every turn also drops slots that came from a now-forgotten fact, in any session (S19, S33). Users can view, correct and forget; "forget everything" lists what will go and waits for a clear yes (S38).

Observability

One append-only JSONL file per session run: state transitions with field diffs, model calls (OpenTelemetry `gen_ai.*` names, tokens, cost, latency), tool calls (args, sanitised result, error class, latency, attempt), approval and outbox transitions, memory events, injection flags, budget hits and errors. Redaction is an allowlist per event type; review text is kept as an excerpt plus a hash; secrets are scrubbed. The console streams the same events live.

Annotated excerpt: timeout after success (`docs/traces/timeout_after_success.jsonl`, turn 4, generated by S10)

```
seq 19 approval  grant   ap_0P47HX8S  by_turn 4             ← the user's "yes" grants the pending approval
seq 20 approval  consume ap_0P47HX8S  fingerprint 7931bb…   ← CAS: granted → consumed on the planned fingerprint
seq 21 outbox    bk_a4cc9c56… → SENT                         ← same transaction, before any network call
seq 23 tool_call book  key bk_a4cc9c56…  latency 3000 ms  error timeout  class UNKNOWN_OUTCOME
seq 24 tool_call get_booking_status → {status: booked, booking_id: B-2041}   ← ask, don't retry
seq 25 outbox    SENT → CONFIRMED  B-2041                    ← only now is success reportable
seq 26 state     executing → done
seq 29 turn_end  "Booked, after one hiccup … It went through once."   claims [B-2041]
```

Exactly one `book` call; the provider's own records show one booking with that key.

Evals

42 scenarios grade end state (bookings, approval, outbox and memory rows), not wording; ten invariants run after each, negative controls prove they bite, and a seeded fuzz adds crashes and clock jumps. The same scenarios replay offline from recorded Claude outputs (`make evals-recorded`); a changed prompt marks them stale. A versioned golden set (184 labelled utterances from SGD, Taskmaster-1, CLINC150, Hinglish-TOP and handwritten rows) gates understanding, with zero false confirmations required. See README for live-model results.

What breaks first at 100×, and what I'd change

1. **Synchronous writes inside a turn.** A 3-second timeout holds a thread and the user lock. Change: commit the outbox row, reply "confirming…", and let a worker pool relay outbox rows and stream results.
2. **SQLite's single writer.** Change: Postgres with `SELECT … FOR UPDATE` on the session row; the approval compare-and-set SQL is unchanged.
3. **Recovery only when a user returns.** Change: a background reconciler sweeps `UNRESOLVED` rows.
4. **No global retry budget.** A provider brown-out is amplified by everyone's retries. Change: a per-provider retry budget and circuit breaker.
5. **A model call on every message.** Change: a deterministic fast path for yes, no and option picks, prompt caching, then model routing measured on recorded traffic.
6. **Local JSONL traces.** Change: export through an OpenTelemetry collector; sample successes, keep every write and error.